

Week 11 Practical

Develop a multithreaded Counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
divya@Divyasree:~$ nano counter.c
divya@Divyasree:~$ gcc counter.c -o counter -pthread
divya@Divyasree:~$ ./counter
Expected counter value: 500000
Actual counter value: 162034
divya@Divyasree:~$ ./counter
./counter
./counter
./counter
Expected counter value: 500000
Actual counter value: 173570
Expected counter value: 500000
Actual counter value: 168331
Expected counter value: 500000
Actual counter value: 183403
Expected counter value: 500000
Actual counter value: 153692
divya@Divyasree:~$ |
```

Code:

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0; // Shared variable

void *increment_counter(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];

    // Create threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    // Wait for all threads to finish
```

```

// Create threads
for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_create(&threads[i], NULL, increment_counter, NULL);
}

// Wait for all threads to finish
for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("Expected counter value: %d\n",
        NUM_THREADS * INCREMENTS);

printf("Actual counter value: %d\n", counter);

return 0;
}

```

Modify the previous program using mutex locks (pthread mutex lock, pthread mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```

divya@Divyasree:~$ nano counter_mutex.c
divya@Divyasree:~$ gcc counter_mutex.c -o counter_mutex -pthread
divya@Divyasree:~$ ./counter_mutex
Expected counter value: 500000
Actual counter value: 500000
divya@Divyasree:~$ ./counter_mutex
./counter_mutex
./counter_mutex
Expected counter value: 500000
Actual counter value: 500000
Expected counter value: 500000
Actual counter value: 500000
Expected counter value: 500000
Actual counter value: 500000
divya@Divyasree:~$ |

```

Code:

```
GNU nano 7.2 counter_mutex.c
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0;

// Declare mutex
pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++)
    {
        // Lock before accessing shared variable
        pthread_mutex_lock(&lock);

        counter++;

        // Unlock after accessing shared variable
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    [ Read 56 lines ]
```

```
GNU nano 7.2 counter_mutex.c
{
    pthread_t threads[NUM_THREADS];

    // Initialize mutex
    pthread_mutex_init(&lock, NULL);

    // Create threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    // Wait for all threads
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
        NUM_THREADS * INCREMENTS);

    printf("Actual counter value: %d\n", counter);

    // Destroy mutex
    pthread_mutex_destroy(&lock);

    return 0;
}
```